

Coding Interviews

Reason before typing

State the expected input and output. Ask about empty input, duplicates, ordering and invalid values before choosing an algorithm.



A small example is useful because it makes hidden assumptions visible. It is easier to correct the contract before coding than to retrofit it into a finished implementation.

What does a successful result look like for an empty input?

Trace the program

Read a short program in execution order. Track the value produced by each expression and the value returned by each function.

```
const numbers = [1, 2, 3];  
const kept = numbers.filter(n => n > 1);  
const doubled = kept.map(n => n * 2);  
console.log(doubled); // [4, 6]
```

A useful trace records both type and value. A result can look semantically related to the task while having the wrong shape for the next operation.

**Try the same reasoning with no matching numbers.
Does your chain still work?**

Test the boundary

Test cases should probe the assumptions, not merely confirm the example used while writing the code.

Include the smallest input, a repeated value and a case that challenges ordering.

```
function firstEven(values) {  
  return values.find(value => value % 2 === 0);  
}  
firstEven([]);           // undefined  
firstEven([3, 5]);       // undefined  
firstEven([3, 4, 2]);    // 4
```

Explain what the caller should do when a result is undefined.

Explain the tradeoff

A technically correct solution is only part of the conversation. Explain the assumptions, the chosen approach and the cost as the input grows.

Avoid claiming that one implementation is faster without defining the input, runtime and measurement. Simpler code can be the better default until a demonstrated bottleneck requires more work.

These are original sample practice notes. No scoring, generated questions or interview simulation is running inside Shelf.

Write one note: what would make you change your chosen solution?